

06

PROGRAMMIERUNG VON WEB-ANWENDUNGEN:

ENTWICKLUNG VON RESTFUL APIS

KOMMUNIKATION ZWISCHEN FRONTEND & BACKEND

WAS SIND RESTFUL APIS?

- REST (Representational State Transfer) ist ein Architekturstil für verteilte Systeme.
- RESTful APIs sind Schnittstellen, die den Zugriff auf Ressourcen über das HTTP-Protokoll ermöglichen.
- Ressourcen können Datenobjekte wie Filme, Benutzer oder Bestellungen sein.
- HTTP-Methoden (GET, POST, PUT, DELETE) werden verwendet, um Operationen auf Ressourcen durchzuführen (lesen, erstellen, aktualisieren, löschen).
- Die Kommunikation erfolgt in der Regel über JSON (JavaScript Object Notation).

ERSTELLUNG VON CONTROLLERN UND MAPPING VON ENDPUNKTEN - 1

- **Controller:** Eine Klasse in Spring Boot, die Anfragen entgegennimmt und verarbeitet.
- **Endpunkte:** URLs, die mit bestimmten HTTP-Methoden und Ressourcen verknüpft sind.
- **Annotations:**
 - `@RestController` : Markiert eine Klasse als Controller.
 - `@GetMapping` , `@PostMapping` , `@PutMapping` , `@DeleteMapping` : Verknüpfen Methoden mit HTTP-Methoden und Endpunkten.
 - `@PathVariable` : Extrahiert Werte aus der URL.
 - `@RequestBody` : Extrahiert Daten aus dem Anfragekörper (meistens JSON).

ERSTELLUNG VON CONTROLLERN UND MAPPING VON ENDPUNKTEN - 2

- **Controller:** Eine Klasse in Spring Boot, die Anfragen entgegennimmt und verarbeitet.
- **Endpunkte:** URLs, die mit bestimmten HTTP-Methoden und Ressourcen verknüpft sind.
- **Annotations:**
 - `@RestController` : Markiert eine Klasse als Controller.
 - `@GetMapping` , `@PostMapping` , `@PutMapping` , `@DeleteMapping` : Verknüpfen Methoden mit HTTP-Methoden und Endpunkten.
 - `@PathVariable` : Extrahiert Werte aus der URL.
 - `@RequestBody` : Extrahiert Daten aus dem Anfragekörper (meistens JSON).

Java

```
@RestController
public class MovieController {

    @GetMapping("/movies")
    public List<Movie> getAllMovies() {
        // ... Logik zum Abrufen aller Filme aus der Datenbank
    }

    @GetMapping("/movies/{id}")
    public Movie getMovieById(@PathVariable Long id) {
        // ... Logik zum Abrufen eines Films anhand der ID
    }

    @PostMapping("/movies")
    public Movie createMovie(@RequestBody Movie movie) {
        // ... Logik zum Erstellen eines neuen Films
    }

    // ... weitere Endpunkte für PUT und DELETE
}
```

UMGANG MIT HTTP-ANFRAGEN UND -ANTWORTEN - 1

○ **Anfragen:**

- Enthalten die HTTP-Methode (GET, POST, etc.), die URL und optional Daten im Anfragekörper.
- Können Header enthalten, die zusätzliche Informationen liefern (z.B. Authentifizierungstoken).

○ **Antworten:**

- Enthalten einen HTTP-Statuscode (z.B. 200 OK, 404 Not Found), Header und optional Daten im Antwortkörper (meistens JSON).
- Der Statuscode gibt an, ob die Anfrage erfolgreich war oder nicht.

○ **Spring Boot:**

- Verwendet Objekte wie `HttpServletRequest` und `HttpServletResponse`, um auf Anfragen und Antworten zuzugreifen.
- Bietet Annotationen wie `@ResponseStatus`, um den Statuscode einer Antwort festzulegen.

UMGANG MIT HTTP-ANFRAGEN UND -ANTWORTEN - 2

- **Anfragen:**

- Enthalten die HTTP-Methode (GET, POST, etc.), die URL und optional Daten im Anfragekörper.
- Können Header enthalten, die zusätzliche Informationen liefern (z.B. Authentifizierungstoken).

- **Antworten:**

- Enthalten einen HTTP-Statuscode (z.B. 200 OK, 404 Not Found), Header und optional Daten im Antwortkörper (meistens JSON).
- Der Statuscode gibt an, ob die Anfrage erfolgreich war oder nicht.

- **Spring Boot:**

- Verwendet Objekte wie `HttpServletRequest` und `HttpServletResponse`, um auf Anfragen und Antworten zuzugreifen.
- Bietet Annotationen wie `@ResponseStatus`, um den Statuscode einer Antwort festzulegen.

Java

```
@GetMapping("/movies/{id}")
public ResponseEntity<Movie> getMovieById(@PathVariable Long id) {
    Optional<Movie> movie = movieRepository.findById(id);
    if (movie.isPresent()) {
        return ResponseEntity.ok(movie.get()); // 200 OK
    } else {
        return ResponseEntity.notFound().build(); // 404 Not Found
    }
}
```

PROJEKT (TEIL 10): REST-ENDPUNKTE IMPLEMENTIEREN

- **Aufgabe:** Implementiere REST-Endpunkte für deine Filmbewertungs-App.
- **Endpunkte:**
 - `GET /movies` : Alle Filme abrufen.
 - `GET /movies/{id}` : Einen Film nach ID abrufen.
 - `POST /movies` : Einen neuen Film erstellen.
 - `PUT /movies/{id}` : Einen Film aktualisieren.
 - `DELETE /movies/{id}` : Einen Film löschen.
 - `POST /movies/{id}/ratings` : Eine Bewertung für einen Film abgeben.
 - `POST /movies/{id}/reviews` : Einen Review für einen Film schreiben (optional).
- **Tipps:**
 - Verwende die richtigen HTTP-Methoden für jede Operation.
 - Gib sinnvolle Statuscodes zurück (200 OK, 201 Created, 404 Not Found, etc.).
 - Verwende `@PathVariable` , um IDs aus der URL zu extrahieren.
 - Verwende `@RequestBody` , um Daten aus dem Anfragekörper zu lesen (JSON).